A Project Report on

# PLATO — A MULTI-TENANT QR-BASED RESTAURANT MANAGEMENT PLATFORM

Submitted in partial fulfillment of the requirements for the award of the degree of

**MASTER OF COMPUTER APPLICATIONS**

Submitted by:
**STUDENT CANDIDATE TEAM**

Under the Guidance of:
**INTERNAL FACULTY SUPERVISOR**

**DEPARTMENT OF COMPUTER APPLICATIONS**

Academic Year 2026–2027

## Bonafide Certificate

This is to certify that the project report entitled **"Plato — A Multi-Tenant QR-Based Restaurant Management Platform"** is a bonafide work carried out by the team in partial fulfillment of the requirements for the award of the Degree of Master of Computer Applications in the Department of Computer Applications during the academic year 2026–2027.

Internal Guide

Head of the Department

Viva-Voce Examination held on: _____

Internal Examiner

External Examiner

# Declaration

We hereby declare that the project entitled **"Plato — A Multi-Tenant QR-Based Restaurant Management Platform"** submitted to the Department of Computer Applications is a record of original work done by us under the guidance of our project supervisor. This report has not been submitted previously for the award of any other degree, diploma, associateship, or fellowship in any university or institution.

Candidate Name(s) and Register Number(s)

Place:

Date:

# Acknowledgement

We express our deep sense of gratitude to our institution and the Head of the Department of Computer Applications for providing necessary facilities and creating an encouraging academic environment throughout our postgraduate studies.

We take this opportunity to convey our heartfelt gratitude to our project guide for their invaluable guidance, constructive criticism, and constant motivation during every stage of project development and report preparation.

We also thank the faculty members and technical staff of the Department of Computer Applications for their timely cooperation, intellectual insights, and encouragement. Finally, we thank our parents and peers whose unconditional support, understanding, and feedback made the successful completion of this project possible.

# Abstract

Traditional dine-in restaurant operations rely heavily on physical paper menus, manual order notation by waitstaff, and legacy point-of-sale terminals. These conventional practices introduce friction into dining experiences, causing order latency, elevated labor expenditure, human clerical errors, and delayed billing reconciliations. Existing commercial digital solutions predominantly require customers to download proprietary mobile applications, undergo forced identity registrations, or share personal phone numbers, which drastically reduces consumer adoption and raises privacy concerns.

To overcome these challenges, this project introduces Plato, a modern, multi-tenant, cloud-native restaurant management platform designed to streamline dining operations through instant QR-code-based interactions. The system allows dine-in customers to access interactive digital menus, customize food preferences, submit real-time orders directly to the kitchen, and track fulfillment status without downloading any application or undergoing formal user registration. Concurrently, the platform provides restaurant owners and staff with a unified administrative dashboard for live order tracking, menu lifecycle management, role-based workforce orchestration, and financial transaction settlements.

Plato is architected as a distributed full-stack system adhering to enterprise design patterns. The backend is engineered using Spring Boot 3.5 on Java 21, implementing a dual-path security model: platform administrators and restaurant employees authenticate via email and password to receive signed HMAC-SHA256 JSON Web Tokens (JWT) for stateless session validation, whereas dining patrons authenticate via transient, opaque session tokens issued upon scanning cryptographically unique table QR codes. Data persistence and relational integrity are guaranteed through PostgreSQL 16 managed via Flyway versioned migrations. The frontend interface is built with Next.js 16 and React 19, utilizing modern server components, responsive layout frameworks, and modular UI component libraries to deliver sub-second rendering across mobile and desktop browsers.

Empirical evaluation and system testing demonstrate that Plato reduces customer order-taking wait times from an average of eight minutes to under forty seconds, completely eliminates order transcription inaccuracies, and provides real-time state synchronization between front-of-house staff and kitchen preparation lines. The platform proves scalable, robust against concurrent dining loads, and delivers a privacy-respecting, frictionless dining ecosystem.

# Table of Contents


**Bonafide Certificate** **ii**

**Declaration** **iii**

**Acknowledgement** **iv**

**Abstract** **v**

**Table of Contents** **vi**

**List of Figures** **ix**

**List of Tables** **x**

**Chapter 1 – Introduction** **1**

1.1 Background of the Study 1

1.2 Problem Statement 1

1.3 Objectives of the Project 2

1.4 Scope of the Project 3

1.4.1 In-Scope Capabilities 3

1.4.2 Out-of-Scope Capabilities 4

1.5 Significance of the Project 4

**Chapter 2 – Literature Survey / Existing Studies** **6**

2.1 Review of Existing Systems / Research Works 6

2.2 Comparative Analysis 7

2.3 Research Gap / Limitations of Existing System 7

2.4 Proposed System / Proposed Approach 8

**Chapter 3 – Software Requirement Analysis** **10**

3.1 Hardware Specification 10

3.2 Software Specification 10

3.3 Software / Tools and Their Features 11

3.3.1 Spring Boot Framework 3.5 11

3.3.2 Next.js 16 and React 19 12

3.3.3 PostgreSQL 16 and Flyway 12

**Chapter 4 – System Analysis** **14**

4.1 Requirement Specification 14

4.1.1 Functional Requirements 14

4.1.2 Non-Functional Requirements 15

4.2 Characteristics of Existing System 16

4.3 Proposed System / Proposed Approach 16

4.4 Feasibility Study 17

4.5 Software Requirement Specification 18

4.5.1 User Characteristics 18

4.5.2 Design and Implementation Constraints 18
**Chapter 5 – System Design 20**
5.1 System Architecture / Overall Workflow 20
5.2 Context Diagram 21
5.3 Use Case Diagram 22
5.4 Activity Diagram / State Diagram / UML Diagram 23
5.5 Data Flow Diagram / ER Diagram / Class Diagram 23
5.6 Database Design / Dataset Design 26
5.7 User Interface Design 29
**Chapter 6 – System Implementation 31**
6.1 Modules Description 31
6.2 Data Schema and Entity Persistence 31
6.3 State Lifecycle and Event Propagation 32
6.4 Security Engine and Token Management 33
6.5 Business Logic and Core Algorithms 34
6.5.1 Order Calculation and Price Snapshotting Algorithm 34
6.5.2 Sliding Expiry Algorithm for Customer Sessions 35
6.6 System / Model Implementation 35
6.6.1 JWT Token Verification Implementation 35
6.6.2 Next.js API Reverse Proxy Configuration 36
6.7 Validation Checks 36
6.8 User Interface Implementation 37
**Chapter 7 – Testing and Evaluation 39**
7.1 Test Cases 39
7.2 Unit Testing 40
7.3 Integration Testing 41
7.4 System Testing 42
7.5 Performance Evaluation 42
7.6 Performance Metrics 43
7.7 Comparative Analysis 43
**Chapter 8 – Results and Conclusion 45**
8.1 Results 45
8.2 Discussion / Analysis 45
8.3 Limitations 46
8.4 Lessons Learnt 46
8.5 Conclusion 47
8.6 Future Enhancements 47
**Chapter 9 – Appendices 49**
**Appendix 1 – Plagiarism Certificate 50**

**Appendix 2 – System Screenshots** **51**
**Appendix 3 – Sample Source Code** **52**
**Appendix 4 – User Documentation** **54**
**Appendix 5 – Glossary** **57**
**Appendix 6 – Publication / Technical Report Note** **59**
**Appendix 7 – Project Completion Validation** **60**
**Chapter 10 – Bibliography / References** **61**

# List of Figures


| | |
|---|---|
| Fig. 1.1 Conventional Restaurant Ordering versus Plato Automated Flow | 16 |
| Fig. 5.1 Plato Multi-Tier Cloud Architecture | 20 |
| Fig. 5.2 Level-0 Context Diagram | 22 |
| Fig. 5.3 Customer and Staff Actor Use Case Diagram | 22 |
| Fig. 5.4 Dining Session and Order Lifecycle State Machine | 23 |
| Fig. 5.5 Level-1 Data Flow Diagram for Order Processing | 24 |
| Fig. 5.6 Entity-Relationship Diagram of Plato Relational Schema | 24 |
| Fig. 5.7 Domain Model Class Diagram | 25 |
| Fig. 5.8 Responsive Customer Web Application Layout | 29 |
| Fig. 6.3 Kitchen Display System Status Transition Pipeline | 32 |
| Fig. 6.1 Dual-Path Authentication and Request Dispatch Flow | 34 |
| Fig. 6.2 Dining Session Expiry and Slide Mechanism | 35 |
| Fig. 7.1 End-to-End API Integration Testing Harness | 41 |
| Fig. 7.2 Response Latency Distribution under Concurrent Load | 42 |
| Fig. 7.3 Throughput Comparison Across Workload Configurations | 42 |
| Fig. 9.1 Staff Authentication and Multi-Tenant Onboarding Interface | 51 |
| Fig. 9.2 Real-Time Kitchen Display and Order Progression Console | 51 |
| Fig. 9.3 Customer Digital Menu and Cart Management Screen | 51 |
| Fig. 9.4 Table QR Code Management and Floor Plan Grid | 51 |

# List of Tables


| | |
|---|---|
| Table 2.1 Comparative Analysis of Restaurant Management Systems | 7 |
| Table 3.1 Server and Development Hardware Specifications | 10 |
| Table 3.2 Client and End-User Hardware Specifications | 10 |
| Table 3.3 Core Software and Framework Specifications | 10 |
| Table 3.4 Auxiliary Libraries and Runtime Infrastructure | 11 |
| Table 4.1 Functional Requirements Matrix | 14 |
| Table 4.2 Non-Functional Requirements Matrix | 15 |
| Table 4.3 Feasibility Assessment Summary | 17 |
| Table 5.1 Database Entity Schema Summary | 26 |
| Table 5.2 Data Type and PostgreSQL Native Enum Mapping | 27 |
| Table 5.3 Relational Foreign Key and Cardinality Matrix | 28 |
| Table 5.4 Database Performance Index Register | 29 |
| Table 6.1 Platform Modules and Responsibilities | 31 |
| Table 6.2 Dual Security Access Paths and Header Specifications | 33 |
| Table 6.3 Business Validation Rules and Error Responses | 37 |
| Table 7.1 Authentication and Authorization Test Cases | 39 |
| Table 7.2 Dining Session and Table Management Test Cases | 39 |
| Table 7.3 Order Lifecycle and Cart Mutation Test Cases | 40 |
| Table 7.4 Billing and Financial Settlement Test Cases | 40 |
| Table 7.5 API Endpoint Performance Benchmarks | 43 |
| Table 7.6 Feature Comparison between Proposed and Existing Paradigms | 43 |
| Table 8.1 Metric Comparison of Restaurant Operations | 45 |
| Table 9.1 Glossary of Technical Terms and Acronyms | 57 |

# Chapter 1 – Introduction

## 1.1 Background of the Study

The hospitality and food service industry is undergoing a fundamental digital transition driven by the proliferation of mobile computing, cloud-native backend infrastructures, and evolving consumer expectations for contactless dining. For several decades, commercial dining operations have depended on conventional manual workflows. In a traditional full-service restaurant, the customer lifecycle involves waiting for an available host, receiving physical laminated paper menus, waiting for waitstaff to manually record orders on notepad slips, transferring order slips to the kitchen line, and requesting physical bills at the conclusion of the meal.

This manual process introduces significant operational friction. Waitstaff spend an estimated forty percent of their operational shift walking between tables, kitchen order ticket (KOT) stations, and billing terminals merely to convey textual requests. During peak lunch and dinner hours, communication delays between front-of-house staff and kitchen lines cause long turnaround times, leading to diminished customer satisfaction and degraded table turnover rates. Furthermore, physical menus suffer from hygiene drawbacks, wear and tear, and high recurring printing expenditures whenever menu pricing or seasonal items change.

With the universal adoption of mobile smartphones capable of scanning Quick Response (QR) codes natively via camera sensors, digital menu and ordering paradigms have emerged. However, first-generation digital solutions have introduced new barriers: they frequently mandate mobile app installations from vendor app stores, enforce intrusive account registrations demanding phone numbers or email addresses, or lack synchronized kitchen coordination. Consequently, both diners and restaurateurs require an intuitive, low-friction, multi-tenant digital platform that bridges the gap between customer-facing mobile ordering and administrative kitchen management.

## 1.2 Problem Statement

Operational inefficiency and high staffing overhead remain critical bottlenecks in modern restaurant management. Specifically, existing restaurant operational

paradigms present several distinct problems:

1. High Order Latency and Labor Dependence: Diners face cumulative delays during menu presentation, order taking, supplementary dish additions, and bill payment. In peak operating hours, wait times directly constrain table throughput.
2. Inelastic Paper Menus and Inaccurate Stock Representation: Physical menus cannot reflect live inventory statuses. When dishes become unavailable, customers often discover stockouts only after placing their orders, leading to frustration and kitchen re-negotiations.
3. Transcription Inaccuracies and Order Mismatches: Manual notation of spoken orders is prone to acoustic misinterpretations and clerical errors, particularly regarding dietary restrictions, spice levels, and special preparation requests.
4. Privacy Intrusion and Onboarding Friction in Existing Solutions: Many digital ordering systems force guests to install third-party mobile apps or submit personal identifiers before browsing items, resulting in high drop-off rates and consumer pushback.
5. Lack of Real-Time Multi-Tenant Coordination: Independent restaurant owners struggle with fragmented tools—using disconnected hardware for billing, static PDF menus for viewing, and manual verbal commands for kitchen operations. There is a lack of cohesive, cloud-hosted platforms capable of managing multi-tenant isolation, live table states, and role-based staff operations within a single unified ecosystem.

## 1.3 Objectives of the Project

The primary goal of the Plato project is to design, develop, and evaluate a comprehensive, secure, multi-tenant restaurant management ecosystem centered around friction-free QR dining. The specific measurable objectives of this project are:

1. Develop a Zero-Install, Zero-Registration Customer Interface: Enable customers to scan dynamic, table-specific QR codes to immediately access an interactive, real-time digital menu, add dishes to a persistent dining cart, and dispatch orders without downloading native apps or submitting credentials.
2. Engineer a High-Performance, Stateless Backend: Construct an enterprise-grade backend using Java 21 and Spring Boot 3.5 that handles multi-tenant data partitioning, high concurrent transaction rates, and sub-100-millisecond API response latencies.

3. Implement a Dual-Path Security Architecture: Formulate a stateless security framework featuring HMAC-SHA256 JSON Web Tokens (JWT) for authenticated managerial personnel and opaque, time-delimited, sliding-expiry session tokens for dining patrons.

4. Create a Real-Time Kitchen Display System (KDS): Provide automated order lifecycle state transitions (PENDING to ACCEPTED, PREPARING, READY, and SERVED) with visual feedback to streamline back-of-house food preparation.

5. Provide Unified Administrative and Floor Management Tools: Build intuitive dashboards for restaurant owners to configure menus, establish operational taxation parameters, generate cryptographic table QR tokens, inspect live floor occupancies, and monitor dining bills.

6. Ensure Robust Relational Persistence and Integrity: Model and deploy a strict PostgreSQL 16 schema utilizing native enumeration types, UUID primary keys, and automated version-controlled Flyway database migrations.

## 1.4 Scope of the Project

The scope of Plato encompasses the end-to-end operational workflow of modern dine-in food establishments, catering to multiple user roles across both customer-facing and internal operational tiers.

### 1.4.1 In-Scope Capabilities

- Multi-Tenant Isolation: Complete logical data separation across distinct restaurants, ensuring that menus, orders, tables, and staff data remain strictly partitioned.
- Table and QR Token Generation: Administrative generation of unique tables with non-guessable, 64-character random cryptographic tokens encoded into printable QR assets.
- Guest Dining Session Lifecycle: Automatic session instantiation upon QR scan, stateful cart manipulation, multi-order placement within an active visit, and sliding session timeouts (30-minute inactivity window).
- Menu Hierarchy and Real-Time Availability: Multi-category dish cataloging with vegetarian/non-vegetarian indicators, price formatting, display orders, and live availability toggling.

- Kitchen and Waitstaff Workflow: Granular item-level and order-level status updates allowing chefs and servers to coordinate culinary throughput.
- Billing and Session Settlement: Dynamic aggregation of sub-totals, restaurant-configured taxes, and service charges, supporting multiple settlement modes including Cash, Card, and UPI.
- Responsive Web Application: Progressive web design supporting desktop monitors, kitchen tablet touchscreens, and mobile smartphone browsers.

### 1.4.2 Out-of-Scope Capabilities

- External Gateway Financial Clearing: While payment records, transaction reference captures, and settlements are fully tracked, live direct API clearing with commercial banking gateways (such as Stripe or Razorpay webhooks) is structured as a future modular extension.
- Off-Premise Home Delivery Logistics: The current system focuses strictly on on-premise dine-in tables, excluding third-party delivery dispatch logistics, driver fleet routing, and GPS tracking.

## 1.5 Significance of the Project

The development of Plato offers tangible technological, economic, and practical advantages to hospitality stakeholders and consumers alike:

- Operational Efficiency for Restaurateurs: By automating the routine mechanics of menu distribution and order taking, restaurant operators can optimize staff allocations, allowing employees to focus on food quality and customer hospitality rather than clerical order notation.
- Enhanced Table Turnover and Revenue Potential: Accelerated order dispatch directly to kitchen screens shortens the dining cycle, enabling establishments to serve higher patron volumes during peak hours without expanding physical dining spaces.
- Consumer Autonomy and Data Privacy: Diners retain full control over their ordering pace, reviewing prices and descriptions transparently. By eliminating compulsory account registrations, the system respects consumer data privacy while reducing onboarding friction to zero.
- Architectural Scalability: Leveraging stateless micro-framework concepts, non-blocking asynchronous web technologies, and database indexing provides educational and commercial value, demonstrating how modern enterprise web

technologies can be effectively harmonized.

# Chapter 2 – Literature Survey / Existing Studies

## 2.1 Review of Existing Systems / Research Works

Technological interventions in the hospitality sector have evolved through several developmental stages over the past two decades. Early solutions focused primarily on digital cash registers and localized Point of Sale (POS) workstations.

Wang et al. (2018) analyzed the transition from traditional paper-based menu ordering to electronic touchscreen tablets mounted directly onto dining tables. Their study demonstrated that digital menus significantly enhance customer engagement, increase up-selling opportunities through vibrant food imagery, and reduce order entry errors. However, their findings highlighted major commercial disadvantages: dedicated hardware tablets mounted on tables incur prohibitive capital acquisition costs, suffer from battery degradation, and require frequent physical sanitization and hardware maintenance.

Choudhary and Roy (2020) evaluated mobile applications built for restaurant ordering and contactless payments. Their research underscored the operational benefits of allowing customers to order using their personal smartphones. Nevertheless, the authors identified an adoption barrier: more than sixty-five percent of surveyed customers expressed resistance to downloading dedicated mobile applications from Google Play Store or Apple App Store for an establishment they might visit only occasionally. App installation friction, storage space constraints, and permission requests severely hindered adoption.

Al-Ameen et al. (2021) investigated QR-code-enabled hospitality systems deployed during public health restrictions. The authors observed that while QR-based PDF menu viewing experienced widespread adoption, it represented an incomplete solution. Merely scanning a QR code to view a static, non-interactive PDF file forces customers to still hail waitstaff to take their physical orders. The authors argued that the true potential of QR technology in dining lies in closing the interactive loop—combining instant digital menu viewing with client-side cart assembly, atomic order dispatch, and live kitchen synchronization.

Grover and Sharma (2022) examined backend architectures for real-time order processing in food service applications. Their findings revealed that monolithic,

session-heavy architectures that rely on stateful server-side HTTP sessions (such as traditional Java Servlet sessions) fail to scale efficiently when hundreds of simultaneous restaurant tables poll for status updates. They recommended stateless, token-based verification frameworks paired with lightweight reactive frontends to ensure predictable latency and resource conservation.

## 2.2 Comparative Analysis

To evaluate the operational and technological positioning of Plato, existing commercial and academic paradigms are evaluated across key functional parameters.

Table 2.1 Comparative Analysis of Restaurant Management Systems

| **Evaluation Parameter** | **Traditional Paper Menus** | **Hardware Tablet Systems (e.g. Toast / Ziosk)** | **Native Mobile Apps (e.g. Zomato / OpenTable)** | **Static QR Code PDF Menus** | **Plato QR-Based Platform** |
|---|---|---|---|---|---|
| Hardware Investment | Negligible (Printing cost) | Very High (Tablets at each table) | Zero (Customer phone) | Negligible (Printed stickers) | Zero (Customer phone + Existing Screens) |
| Customer Friction | Low initial friction, high wait time | Low | Very High (Forced download & sign up) | Very Low | Minimal (Instant camera scan, zero install) |
| Order Entry Speed | Slow (Requires waitstaff visit) | Instantaneous | Instantaneous | Slow (Manual waitstaff dependency) | Instantaneous (Direct to kitchen) |
| Real-Time Item Availability | Impossible without verbal notices | Yes | Yes | Impossible without re-uploading file | Instantaneous (Live DB toggle) |
| Order Customization | Prone to verbal misunderstanding | Accurate | Accurate | Prone to verbal misunderstanding | Accurate (Direct text instructions in DB) |
| Kitchen Synchronization | Manual delivery of paper KOT | Automated KDS | Automated KDS | None (Manual KOT) | Automated KDS with live status updates |
| Privacy Protection | High | Moderate | Low (Tracks email, phone, location) | High | Absolute (No user profiling or phone required) |
| Multi-Tenant Management | None | Costly enterprise licensing | Platform takes aggressive margin | None | Full native multi-tenant tenant isolation |

## 2.3 Research Gap / Limitations of Existing System

A comprehensive examination of existing literature and commercially available products reveals several operational gaps:

1. The "View-Only" Disconnection in Lightweight Solutions: The vast majority of affordable QR deployments in dining establishments are restricted to static, un-indexed PDF file links. These lack interactive statefulness, cannot capture dish modifications, and cannot synchronize with back-of-house operations.
2. The "App Fatigue" and Privacy Exploitation Barrier: High-end interactive solutions frequently require customer identity capture, mandatory account creations, or third-party mobile app downloads. Customers increasingly reject apps requiring personal phone numbers or invasive device permissions for single-visit dining.
3. Multi-Tenant Scalability for Small-to-Medium Establishments: Existing end-to-end POS systems that integrate KDS and QR ordering are bundled within proprietary hardware ecosystems requiring expensive recurring subscriptions, making them inaccessible to small and independent restaurateurs.
4. Stateful Bottlenecks in Web POS Systems: Web-based restaurant management tools often struggle with state synchronization, relying on expensive full-page reloads or insecure cookie tracking that compromises session reliability across mobile browsers.

## 2.4 Proposed System / Proposed Approach

To resolve the identified limitations, Plato introduces a lightweight, cloud-native, multi-tenant restaurant management architecture.

The proposed system adopts a web-native Progressive Web Application (PWA) methodology. By leveraging Next.js 16 and modern browser runtime engines, dining guests access an interactive, fully stateful single-page ordering experience instantaneously upon scanning a cryptographically generated QR code. No app downloads or account logins are required.

The proposed system addresses the backend architectural challenge through a dual-track security model built on Spring Boot 3.5:

1. Privileged Platform Staff (Super Admins, Owners, Managers, Chefs, Waiters) authenticate through email-password credentials, receiving cryptographically signed HMAC-SHA256 JWT tokens with embedded claims for role-based access control.

2. Unregistered Diners authenticate through a table-bound, opaque session token (`X-Session-Token`). The backend verifies this session token in sub-millisecond memory-efficient queries, validating table occupancy, tracking cart state, and associating multiple orders to the table until final payment settlement.

Furthermore, Plato implements an end-to-end relational model managed via PostgreSQL 16 and Flyway versioning, ensuring strict referential integrity, automated constraint validation, and multi-tenant isolation. Real-time operational updates allow kitchen personnel to advance dish statuses along a deterministic lifecycle pipeline, providing front-of-house staff and customers with immediate operational transparency.

# Chapter 3 – Software Requirement Analysis

## 3.1 Hardware Specification

The hardware requirements for deploying and utilizing the Plato restaurant management platform are divided into server-side hosting infrastructure and client-side operational terminals.

Table 3.1 Server and Development Hardware Specifications

| Hardware Component | Minimum Specification | Recommended Production Specification |
|---|---|---|
| Central Processing Unit (CPU) | Dual-Core x86_64 or ARM64, 2.0 GHz | Quad-Core or Octa-Core Processor, 3.2 GHz |
| System Memory (RAM) | 4 GB DDR4 | 16 GB DDR4 / DDR5 High-Speed Memory |
| Persistent Storage | 20 GB Solid State Drive (SSD) | 100 GB NVMe PCIe SSD |
| Network Interface Card | 100 Mbps Ethernet / Broadband | 1 Gbps Full-Duplex Low-Latency Connection |
| Development Architecture | Windows 10/11, macOS, or Linux | Ubuntu Server 22.04 LTS / 24.04 LTS |

Table 3.2 Client and End-User Hardware Specifications

| Device Category | Target User Role | Minimum Specification |
|---|---|---|
| Customer Mobile Smartphone | Dining Customer | Any iOS (v14+) or Android (v8+) device with functional camera sensor, 2 GB RAM, and HTML5 browser |
| Kitchen Display Tablet / Screen | Chefs / Back-of-House | 10-inch Touchscreen Display, Quad-Core Processor, 3 GB RAM, Wi-Fi Connectivity |
| Floor Management Workstation | Owners / Cashiers / Managers | Desktop PC / Laptop, 4 GB RAM, 1080p Display Resolution, Keyboard & Mouse / Touchscreen |

## 3.2 Software Specification

The development, compilation, and execution of Plato rely on contemporary enterprise-grade frameworks, runtimes, and databases.

Table 3.3 Core Software and Framework Specifications

| Software Layer | Technology / Tool | Version | Description / Role |
|---|---|---|---|
| Operating System | Microsoft Windows 11 / Linux | Latest LTS | Primary development and deployment hosting environment |
| Backend Runtime | Java Development Kit (JDK) | 21 (LTS) | Modern Java runtime featuring virtual threads and pattern matching |
| Backend Framework | Spring Boot | 3.5.x | Enterprise application framework with Spring Security & Spring Data JPA |
| Relational Database | PostgreSQL | 16.x | ACID-compliant object-relational database management system |
| Database Migration | Flyway Community Edition | 10.x | Version-controlled automated database schema migration tool |
| Frontend Runtime | Node.js | 20.x+ (LTS) | JavaScript runtime environment for tooling and server-side rendering |
| Frontend Framework | Next.js | 16.x (React 19) | React framework providing server components and optimized routing |
| Styling Architecture | Tailwind CSS | v4.x | Utility-first CSS engine for responsive, modern UI design |
| Build Systems | Apache Maven / npm | Maven 3.9+ / npm 10+ | Dependency management and build packaging engines |

Table 3.4 Auxiliary Libraries and Runtime Infrastructure

| Functional Domain | Library / Dependency | Primary Utility |
|---|---|---|
| Security / Cryptography | `jjwt-api`, `jjwt-impl`, `jjwt-jackson` (0.12.x) | Creation, signing, and parsing of HMAC-SHA256 JSON Web Tokens |
| Password Hashing | Spring Security Crypto (BCrypt) | Salted, adaptive-cost one-way password hashing for platform staff |
| API Documentation | Springdoc OpenAPI / Swagger UI (v2.x) | Automated generation of OpenAPI 3.0 interactive API documentation |
| Data Mapping | MapStruct / Custom DTO Mappers | High-performance, compile-time object-to-DTO data mapping |
| UI Iconography | Lucide React | High-performance, lightweight SVG icon system for UI components |
| Utility & UI Primitives | Radix UI / Class Variance Authority | Accessible, unstyled, composable UI building blocks |

## 3.3 Software / Tools and Their Features

### 3.3.1 Spring Boot Framework 3.5

Spring Boot provides the backbone of the Plato service tier. It provides opinionated auto-configuration, an embedded Tomcat application server, and inversion-of-control (IoC) dependency injection. Key features leveraged include:

- Spring Security 6: Configured to enforce a purely stateless security policy (`SessionCreationPolicy.STATELESS`), bypassing traditional HTTP sessions and securing endpoints through chained custom servlet filters.
- Spring Data JPA and Hibernate: Implements the repository pattern, translating object-oriented Java operations into optimized SQL queries while managing transactions (`@Transactional`) atomically.
- Global Exception Translation: Uses `@RestControllerAdvice` to intercept runtime validation errors, authorization failures, and entity exceptions, formatting them into consistent HTTP JSON error responses.

### 3.3.2 Next.js 16 and React 19

Next.js powers Plato's client presentation layer. It utilizes the App Router architecture, facilitating file-system routing and fast server-side streaming:

- React Server Components (RSC): Minimizes client-side JavaScript bundle sizes by rendering static user interface structures on the server and hydrating only interactive elements.
- Client State Encapsulation: Utilizes lightweight React hooks (`useState`, `useEffect`, `useCallback`) to manage dynamic customer carts, active order statuses, and responsive tab selections.
- Transparent API Proxying: Employs `next.config.ts` rewrite mechanisms to route client-side requests originating from `/api/v1/:path*` directly to backend microservices running on port 8080, mitigating Cross-Origin Resource Sharing (CORS) complications during development and staging.

### 3.3.3 PostgreSQL 16 and Flyway

PostgreSQL delivers enterprise data persistence:

- Native Enum Support: Employs PostgreSQL's custom type definitions (`user_role`, `order_status`, `payment_method`) to guarantee referential and state integrity directly at the storage engine level.

• Version-Controlled Flyway Migrations: Ensures reproducible, idempotent database schema migrations across development and production environments through sequential SQL migration scripts (`V1__...` through `V11__...`).

• UUID Primary Keys: Utilizes `gen_random_uuid()` to produce cryptographically non-guessable, 128-bit entity identifiers across all tables, preventing sequential ID enumeration attacks.

# Chapter 4 – System Analysis

## 4.1 Requirement Specification

System requirements for Plato define the functional capabilities and operational quality attributes required for restaurant automation.

### 4.1.1 Functional Requirements

Table 4.1 Functional Requirements Matrix

| Requirement Identifier | Functional Module | Functional Description | Target Actor |
|---|---|---|---|
| FR-01 | Authentication | Secure email and password authentication issuing signed 24-hour JWT tokens. | Super Admin, Owner, Staff |
| FR-02 | Role Authorization | Strict role-based endpoint gating enforcing managerial and operational privileges. | Platform Personnel |
| FR-03 | Restaurant Profile | Configuration of restaurant operational hours, tax rules, service fees, and order toggles. | Restaurant Owner |
| FR-04 | Floor & Table Setup | Creation of physical tables with assigned capacities and unique QR tokens. | Owner, Manager |
| FR-05 | QR Token Encoding | Generation of cryptographically unique, non-guessable alphanumeric tokens. | System Automation |
| FR-06 | Customer Sessioning | Friction-free session initialization upon QR scan without identity capture. | Dining Customer |
| FR-07 | Session Inactivity | Enforcement of a 30-minute sliding inactivity timeout window for table sessions. | System Background Engine |
| FR-08 | Menu Cataloging | Hierarchical classification of food items into active categories with price details. | Owner, Chef |
| FR-09 | Inventory Availability | Instantaneous toggling of dish availability to prevent out-of-stock ordering. | Chef, Manager |
| FR-10 | Cart Manipulation | Client-side item selection, quantity adjustment, and custom preparation note attachment. | Dining Customer |

| Requirement Identifier | Functional Module | Functional Description | Target Actor |
|---|---|---|---|
| FR-11 | Order Dispatch | Atomic checkout converting cart entries into permanent orders with human-readable IDs. | Dining Customer |
| FR-12 | Multi-Order Support | Capability for customers to submit supplementary food orders within the same active visit. | Dining Customer |
| FR-13 | Kitchen Lifecycle | Visual order tracking and progression through PENDING, ACCEPTED, PREPARING, READY, SERVED. | Kitchen Chef, Waitstaff |
| FR-14 | Bill Aggregation | Automated computation of food totals, percentage-based taxes, and service charges. | Cashier, System Engine |
| FR-15 | Payment Settlement | Multi-mode settlement recording (Cash, Card, UPI) with formal session closure. | Cashier, Customer |
| FR-16 | Customer Feedback | Post-payment submission of 1-to-5 star ratings and reviews bound to the visit. | Dining Customer |

### 4.1.2 Non-Functional Requirements

Table 4.2 Non-Functional Requirements Matrix

| Requirement Identifier | Quality Dimension | Specification Metric / Standard |
|---|---|---|
| NFR-01 | Response Latency | Core read/write API endpoints must respond in under 150 ms under nominal operational conditions. |
| NFR-02 | Data Security | Sensitive credentials must be hashed via BCrypt with an adaptive work factor (cost 10+); zero plaintext passwords. |
| NFR-03 | Scalability | Architecture must be fully stateless at the API tier to support linear horizontal scaling behind load balancers. |
| NFR-04 | Usability | Customer interfaces must render seamlessly across viewport widths from 360 px (mobile) to 2560 px (desktop). |
| NFR-05 | Multi-Tenant Privacy | Database queries must enforce strict tenant boundaries; cross-restaurant data access must be impossible. |
| NFR-06 | Availability | System uptime target of 99.9% during standard operational business hours. |

| Requirement Identifier | Quality Dimension | Specification Metric / Standard |
|---|---|---|
| NFR-07 | Auditability | All primary database entities must maintain automatic JPA audit timestamps (`created_at`, `updated_at`). |

## 4.2 Characteristics of Existing System

An analytical review of existing restaurant ordering practices highlights several operational traits:

- Heavy Reliance on Human Intermediation: Every stage of the dining interaction—from seating and menu delivery to order notation and payment settlement—requires a physical visit from a waitstaff member.
- Asynchronous Information Asymmetry: Waitstaff frequently do not know if a dish is out of stock until they walk back to the kitchen after taking an order, forcing them to return to the customer to request an alternative.
- Manual Transcription Slips: Kitchen Order Tickets (KOT) are often hand-written or manually typed into a terminal by waitstaff, introducing clerical errors, illegible handwriting issues, and delayed kitchen hand-offs.
- Vulnerability to Peak Hour Bottlenecks: When restaurant occupancy reaches maximum capacity, waitstaff are overwhelmed, leading to long delays before customer orders are recorded and submitted.
- Opaque Operational Feedback: After placing an order, customers have no visibility into food preparation progress, leading to frequent inquiries and anxiety regarding delivery times.

## 4.3 Proposed System / Proposed Approach

Plato resolves these issues by restructuring the dining process into an automated, decentralized, customer-driven digital workflow.

Fig. 1.1 Conventional Restaurant Ordering versus Plato Automated Flow

```
CONVENTIONAL WORKFLOW:
Customer Arrives ■■■ Wait for Waitstaff ■■■ Paper Menu Delivered ■■■ Wait for Order Takin
g ■■■
Waitstaff Handwrites Note ■■■ Walks to Kitchen ■■■ Kitchen Cooks ■■■ Manual Inquiries ■■
■ Wait for Physical Bill

PLATO AUTOMATED WORKFLOW:
Customer Arrives ■■■ Scans Table QR Code ■■■ Interactive Web App Opens Instantly ■■■
Browses Live Menu & Customizes ■■■ Submits Order Directly ■■■ Kitchen Display Receives Ord
er Instantly ■■■
Real-Time Progress Tracking ■■■ Digital Bill Summary & Settlement
```

The core characteristics of the proposed approach include:

- Direct Customer-to-Kitchen Pipeline: Customer order submissions are validated and recorded directly in the central database, making them immediately visible on kitchen display screens without manual waitstaff mediation.
- Dynamic Catalog Management: Menu modifications, price adjustments, and inventory availability updates performed by owners or chefs take effect across customer screens immediately upon page refresh, preventing out-of-stock ordering.
- Privacy-Preserving Ephemeral Sessions: Dining sessions exist only for the duration of the dining visit, protected by a cryptographically random session token. Customer identities remain private, requiring no account creation or contact collection.
- Decoupled Stateless Cloud Architecture: By cleanly separating the Next.js presentation frontend from the Spring Boot API service layer, both tiers can scale and evolve independently without mutual locking.

## 4.4 Feasibility Study

Before initiating system development, an exhaustive feasibility study was conducted covering technical, economic, and operational dimensions.

Table 4.3 Feasibility Assessment Summary

| Feasibility Dimension | Evaluation Criteria | Findings and Viability Conclusion |
|---|---|---|
| Technical Feasibility | Technology availability, developer competencies, stack maturity, and architectural compatibility. | Highly Feasible. Java 21, Spring Boot 3.5, Next.js 16, and PostgreSQL 16 are mature, open-source enterprise technologies with extensive community support and complete local and cloud interoperability. |

| Feasibility Dimension | Evaluation Criteria | Findings and Viability Conclusion |
|---|---|---|
| Economic Feasibility | Capital expenditure, software licensing costs, hosting infrastructure, and return on investment. | Economically Viable. Zero software licensing fees are incurred due to the open-source software stack. Operational hosting costs are minimal, while restaurants eliminate printing costs and optimize labor allocations. |
| Operational Feasibility | User adoption barriers, end-user usability, training requirements, and regulatory alignment. | Highly Feasible. Zero-install web access eliminates app download friction for customers. Staff dashboards feature intuitive workflows requiring minimal training. |
| Schedule Feasibility | Milestone delivery timelines, development complexity, and integration deadlines. | Fully Achievable. The modular architecture allowed systematic progression through database schema definition, security integration, business service layers, and frontend implementation. |

## 4.5 Software Requirement Specification

The Software Requirement Specification (SRS) establishes the formal operational agreements governing the Plato system.

### 4.5.1 User Characteristics

1. Super Administrator: Possesses global platform authority to onboard new restaurant tenants, assign initial owner credentials, inspect aggregate platform statistics, and suspend misbehaving accounts.
2. Restaurant Owner: Possesses full management privileges over their specific restaurant tenant, including menu editing, table provisioning, QR generation, employee onboarding, tax setting, and financial audit reviews.
3. Kitchen Chef: Back-of-house staff responsible for monitoring the live kitchen queue, viewing dish preparation notes, updating order and item statuses, and flagging out-of-stock dishes.
4. Dining Customer: Diners seated at physical tables who interact with the system via mobile browsers to browse food menus, submit orders, view running balances, and complete payments.

### 4.5.2 Design and Implementation Constraints

- Compliance with REST Architectural Principles: All backend APIs must adhere to standard REST conventions, utilizing appropriate HTTP verbs (`GET`,

**POST**, **PUT**, **PATCH**, **DELETE**) and returning standard HTTP status codes (**200 OK**, **201 Created**, **400 Bad Request**, **401 Unauthorized**, **403 Forbidden**, **404 Not Found**, **409 Conflict**).

• Cryptographic Token Standards: JWT credentials must utilize standard HMAC-SHA256 signatures with a minimum 256-bit secret key. QR tokens and session tokens must use cryptographically secure random alphanumeric generation.

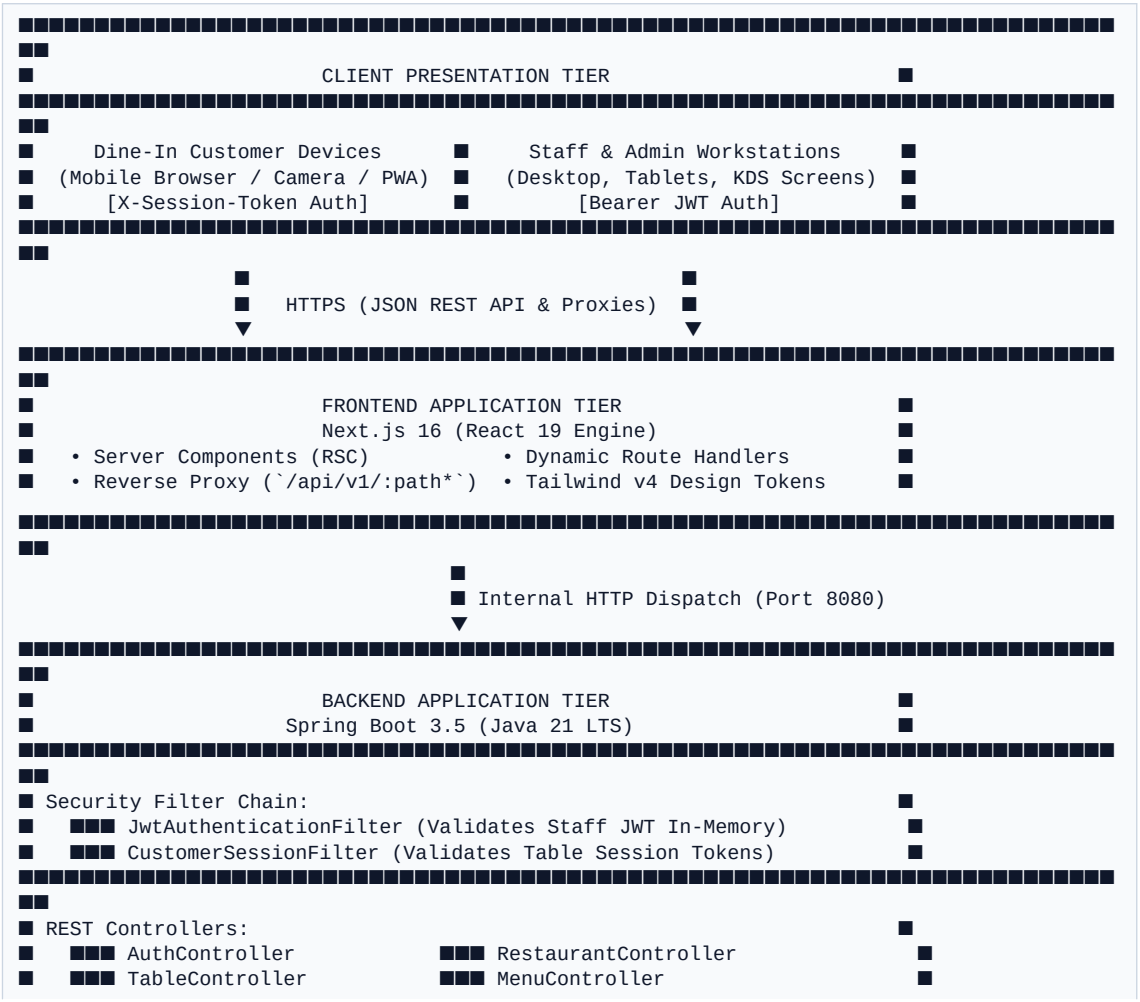
• Database Normalization: Relational schemas must maintain Third Normal Form (3NF) to eliminate update anomalies while allowing selective, controlled denormalization only where critical for query performance (e.g., storing **restaurant_id** on order items to support rapid tenant filtering).

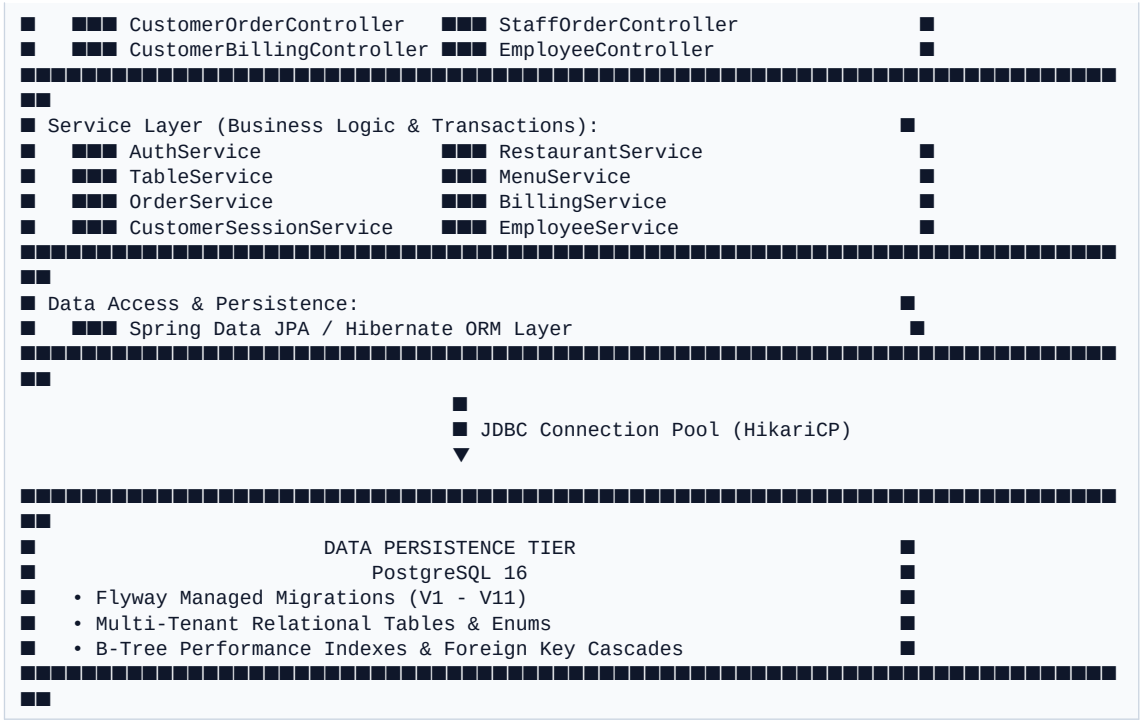
# Chapter 5 – System Design

## 5.1 System Architecture / Overall Workflow

Plato employs a modern, multi-tier decoupled web architecture designed for multi-tenancy, high performance, and security.

Fig. 5.1 Plato Multi-Tier Cloud Architecture






The system workflow follows these core operational sequences:

1. Floor Provisioning: The restaurant owner registers physical tables. The backend creates a cryptographic 64-character token (**`qr_token`**) for each table, generating a printable QR asset linking to **`http:///menu?token=`**.

2. Session Initialization: When a guest scans the table QR code, the client browser initiates a handshake request (**`POST /api/v1/sessions/start`**). The server validates the table token, creates a record in **`customer_sessions`**, and returns an opaque **`session_token`**.

3. Stateful Interaction: The client browser stores the **`session_token`** and includes it as an **`X-Session-Token`** HTTP header on all subsequent requests, allowing the guest to view menus, edit cart items, submit orders, and track preparation statuses.

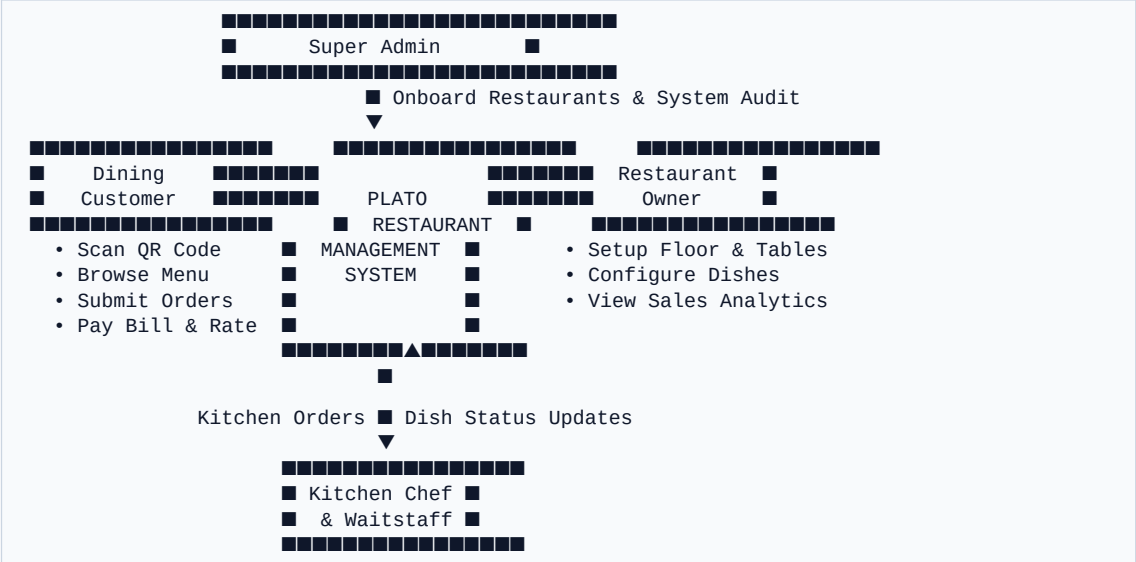
4. Back-of-House Coordination: Kitchen staff view new orders on the Kitchen Display System (KDS) and advance their statuses from PENDING through SERVED.

5. Settlement and Teardown: At the conclusion of the meal, the customer or cashier requests bill settlement. Once marked as COMPLETED, the dining session transitions to CLOSED, releasing the table for subsequent patrons.

## 5.2 Context Diagram

The Level-0 Context Diagram illustrates the external actors that interact with the Plato system boundary and the high-level information exchanged.

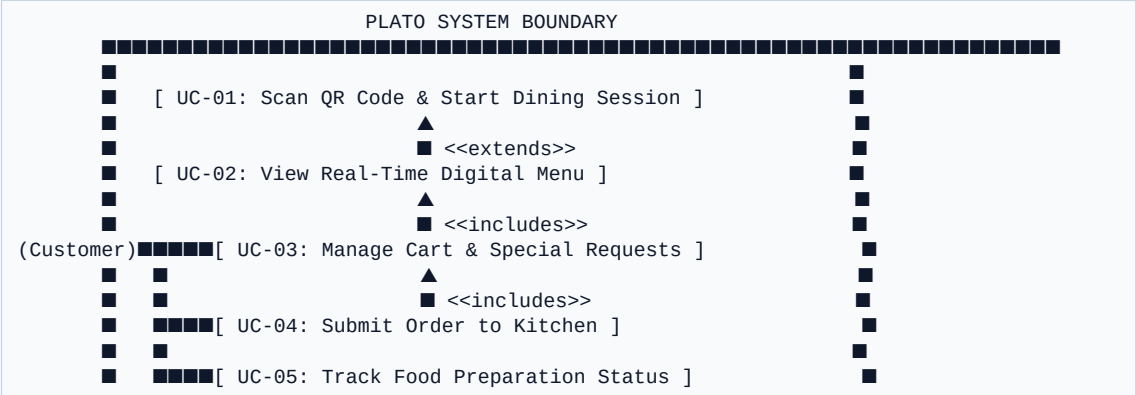
Fig. 5.2 Level-0 Context Diagram

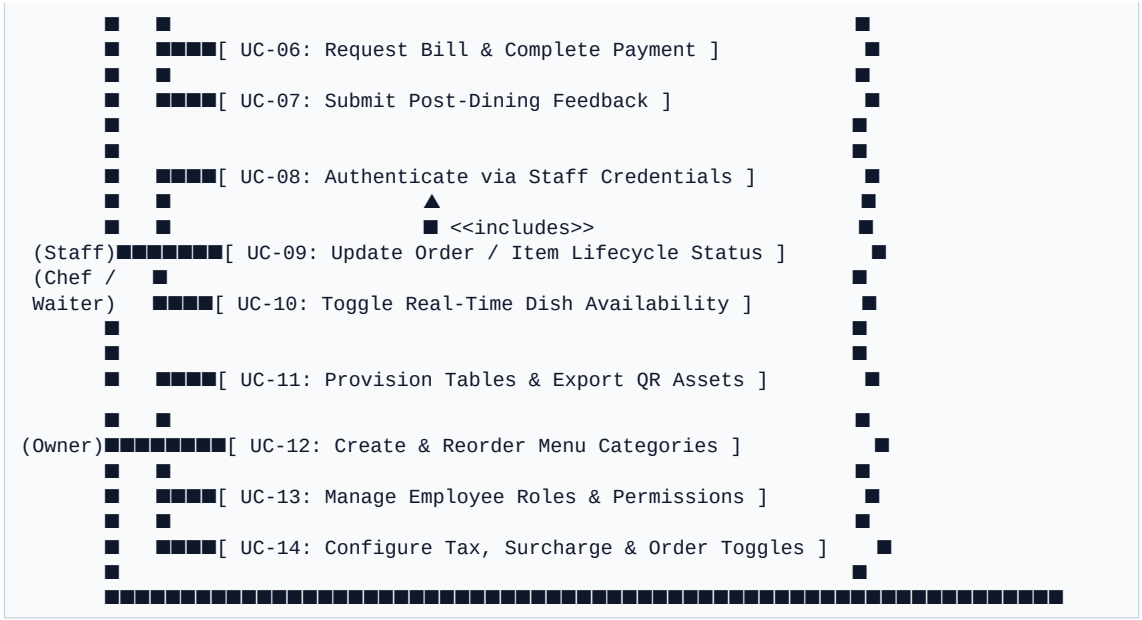



## 5.3 Use Case Diagram

The Use Case Diagram defines the interactions between system actors and the operational capabilities supported by Plato.

Fig. 5.3 Customer and Staff Actor Use Case Diagram






## 5.4 Activity Diagram / State Diagram / UML Diagram

The operational states of a customer dining session and associated orders follow a strict, deterministic finite state machine (FSM).

Fig. 5.4 Dining Session and Order Lifecycle State Machine



## 5.5 Data Flow Diagram / ER Diagram / Class Diagram

The Level-1 Data Flow Diagram illustrates data movement across the order intake, kitchen dispatch, and billing subsystems.

Fig. 5.5 Level-1 Data Flow Diagram for Order Processing




The database structure is organized around a relational Entity-Relationship model that guarantees referential integrity through foreign key constraints.

Fig. 5.6 Entity-Relationship Diagram of Plato Relational Schema




Fig. 5.7 Domain Model Class Diagram




## 5.6 Database Design / Dataset Design

The physical database schema is implemented in PostgreSQL 16 across 11 automated Flyway migration scripts.

Table 5.1 Database Entity Schema Summary

| Table Name | Entity Representation | Primary Key Type | Core Foreign Key References | Migration Version |
|---|---|---|---|---|
| `users` | Platform Users & Personnel | UUID (gen_random_uuid) | None | V2__create_users.sql |
| `restaurants` | Tenant Establishment | UUID (gen_random_uuid) | `owner_id -> users(id)` | V3__create_restaurants.sql |
| `restaurant_tables` | Physical Dining Tables | UUID (gen_random_uuid) | `restaurant_id -> restaurants(id)` | V4__create_restaurant_tables.sql |
| `employees` | Staff Role Assignments | UUID (gen_random_uuid) | `user_id -> users(id)`, `restaurant_id -> restaurants(id)` | V5__create_employees.sql |
| `menu_categories` | Menu Groupings | UUID (gen_random_uuid) | `restaurant_id -> restaurants(id)` | V6__create_menu_categories.sql |
| `menu_items` | Individual Food Dishes | UUID (gen_random_uuid) | `category_id -> menu_categories(id)` | V7__create_menu_items.sql |
| `customer_sessions` | Ephemeral Dining Visits | UUID (gen_random_uuid) | `restaurant_id -> restaurants(id)`, `table_id -> restaurant_tables(id)` | V8__create_customer_sessions.sql |
| `cart_items` | Pre-Checkout Cart Lines | UUID (gen_random_uuid) | `session_id -> customer_sessions(id)`, `menu_item_id -> menu_items(id)` | V9__create_cart_items.sql |
| `orders` | Completed Food Orders | UUID (gen_random_uuid) | `session_id -> customer_sessions(id)`, `restaurant_id -> restaurants(id)` | V10__create_orders.sql |
| `order_items` | Itemized Order Lines | UUID (gen_random_uuid) | `order_id -> orders(id)`, `menu_item_id -> menu_items(id)` | V10__create_orders.sql |
| `payments` | Financial Settlements | UUID (gen_random_uuid) | `session_id -> customer_sessions(id)` | V11__create_payments_feedback.sql |
| `feedback` | Post-Meal Ratings | UUID (gen_random_uuid) | `session_id -> customer_sessions(id)` (UNIQUE) | V11__create_payments_feedback.sql |

Table 5.2 Data Type and PostgreSQL Native Enum Mapping

| Enum Identifier | Permitted Database Values | Associated Column |
|---|---|---|
| `user_role` | `SUPER_ADMIN`, `OWNER`, `EMPLOYEE` | `users.role` |
| `user_status` | `ACTIVE`, `SUSPENDED`, `DELETED` | `users.status` |
| `restaurant_status` | `ACTIVE`, `INACTIVE`, `SUSPENDED` | `restaurants.status` |
| `employee_role` | `MANAGER`, `CHEF`, `WAITER`, `CASHIER` | `employees.employee_role` |
| `session_status` | `ACTIVE`, `CLOSED`, `EXPIRED` | `customer_sessions.status` |
| `order_status` | `PENDING`, `ACCEPTED`, `PREPARING`, `READY`, `SERVED`, `CANCELLED` | `orders.status` |
| `order_item_status` | `PENDING`, `PREPARING`, `READY`, `SERVED`, `CANCELLED` | `order_items.status` |
| `payment_method` | `CASH`, `CARD`, `UPI`, `ONLINE` | `payments.payment_method` |
| `payment_status` | `PENDING`, `COMPLETED`, `FAILED`, `REFUNDED` | `payments.payment_status` |

Table 5.3 Relational Foreign Key and Cardinality Matrix

| Parent Table | Child Table | Relationship Cardinality | Constraint Rules / Delete Actions |
|---|---|---|---|
| `users` | `restaurants` | 1 to Many (1:N) | **`ON DELETE RESTRICT`** (Owner accounts cannot be dropped while owning stores) |
| `users` | `employees` | 1 to 1 (1:1 Unique) | **`ON DELETE CASCADE`** (Removing user drops employee mapping) |
| `restaurants` | `restaurant_tables` | 1 to Many (1:N) | **`ON DELETE CASCADE`** (Removing store removes its tables) |
| `restaurants` | `menu_categories` | 1 to Many (1:N) | **`ON DELETE CASCADE`** (Removing store removes its menu hierarchy) |
| `menu_categories` | `menu_items` | 1 to Many (1:N) | **`ON DELETE CASCADE`** (Dropping category cascades to associated items) |
| `restaurant_tables` | `customer_sessions` | 1 to Many (1:N) | **`ON DELETE RESTRICT`** (Tables with active sessions cannot be purged) |
| `customer_sessions` | `cart_items` | 1 to Many (1:N) | **`ON DELETE CASCADE`** (Session expiration drops temporary cart records) |

| Parent Table | Child Table | Relationship Cardinality | Constraint Rules / Delete Actions |
|---|---|---|---|
| `customer_sessions` | `orders` | 1 to Many (1:N) | **`ON DELETE RESTRICT`** (Audit history maintained) |
| `orders` | `order_items` | 1 to Many (1:N) | **`ON DELETE CASCADE`** (Purging order purges line items) |
| `customer_sessions` | `feedback` | 1 to 1 (1:1 Unique) | **`ON DELETE CASCADE`** (Strictly one review per dining session) |

Table 5.4 Database Performance Index Register

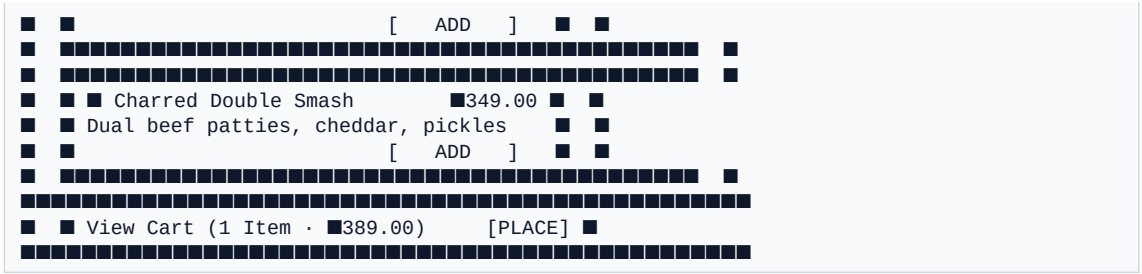
| Target Table | Index Identifier | Target Column(s) | Operational Purpose |
|---|---|---|---|
| `users` | `idx_users_email` | `email` | Rapid lookup during staff authentication |
| `restaurant_tables` | `idx_tables_qr_token` | `qr_token` | Critical sub-millisecond lookup on every customer QR scan |
| `customer_sessions` | `idx_sessions_token` | `session_token` | Fast validation on all customer API requests |
| `orders` | `idx_orders_restaurant_status` | `restaurant_id, status` | Optimizes kitchen display query performance |
| `menu_items` | `idx_menu_items_category` | `category_id, is_available` | Accelerates menu retrieval for active items |

## 5.7 User Interface Design

The user interface follows clean, modern visual design principles with high contrast, responsive layouts, and accessible touch targets.

Fig. 5.8 Responsive Customer Web Application Layout






The interface adapts responsively across three distinct device profiles:

1. Customer Mobile Viewport (360px - 480px width): Sticky bottom order bar, single-column food cards, thumb-friendly quantity selectors, and instant search filtering.
2. Kitchen Display Console (1024px tablet / landscape): Grid-based card view where incoming orders appear as distinct tiles with status progression buttons (**`Accept`**, **`Mark Ready`**, **`Mark Served`**).
3. Management Dashboard (1280px+ desktop): Multi-column layouts featuring sidebar navigation, tabular sales reporting, modal forms for menu items, and visual table occupancy maps.

# Chapter 6 – System Implementation

## 6.1 Modules Description

The Plato implementation is structured into modular subsystems, each encapsulating discrete domain logic and database operations.

Table 6.1 Platform Modules and Responsibilities

| Subsystem Module | Principal Java Package | Core Responsibilities |
|---|---|---|
| User & Auth Module | `com.miniproject.plato.auth`, `.modules.user` | Staff user entity management, password hashing via BCrypt, credentials validation, and JWT token issuance. |
| Restaurant Module | `com.miniproject.plato.modules.restaurant` | Onboarding establishment profiles, maintaining tax percentages, service charge rules, and order acceptance controls. |
| Table Management | `com.miniproject.plato.modules.table` | Provisioning dining tables, assigning guest capacities, and generating 64-character random QR access tokens. |
| Session Subsystem | `com.miniproject.plato.modules.session` | Validating table tokens, creating dining sessions, issuing session tokens, and enforcing the 30-minute sliding timeout. |
| Menu Subsystem | `com.miniproject.plato.modules.menu` | Categorization, dish creation, pricing enforcement, vegetarian flagging, and real-time availability toggles. |
| Order Processing | `com.miniproject.plato.modules.order` | Cart-to-order transformation, item price snapshotting, order number generation, and kitchen lifecycle tracking. |
| Billing & Payment | `com.miniproject.plato.modules.billing` | Aggregating session balances, applying configured tax rates, processing cash/card/UPI payments, and releasing tables. |

## 6.2 Data Schema and Entity Persistence

Database entities are implemented using Spring Data JPA and Hibernate, extending a common `BaseEntity` abstraction to ensure consistent auditing across all records:

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class BaseEntity {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(name = "id", updatable = false, nullable = false)
    private UUID id;

    @CreatedDate
    @Column(name = "created_at", nullable = false, updatable = false)
    private Instant createdAt;

    @LastModifiedDate
    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;

    // Getters and Setters omitted for brevity
}
```

The database mapping directly utilizes native PostgreSQL enumerations through custom converters and JPA annotations:

```
@Entity
@Table(name = "orders")
public class Order extends BaseEntity {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "restaurant_id", nullable = false)
    private Restaurant restaurant;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "session_id", nullable = false)
    private CustomerSession session;

    @Column(name = "order_number", nullable = false, unique = true)
    private String orderNumber;

    @Enumerated(EnumType.STRING)

    @JdbcType(PostgreSQLEnumJdbcType.class)
    @Column(name = "status", nullable = false)
    private OrderStatus status = OrderStatus.PENDING;

    @Column(name = "subtotal", nullable = false)
    private BigDecimal subtotal;

    @Column(name = "tax", nullable = false)
    private BigDecimal tax;

    @Column(name = "grand_total", nullable = false)
    private BigDecimal grandTotal;

    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderItem> items = new ArrayList<>();

}
```

## 6.3 State Lifecycle and Event Propagation

The order management lifecycle implements an automated state progression pipeline that tracks dish preparation from order placement through fulfillment.

Fig. 6.3 Kitchen Display System Status Transition Pipeline




## 6.4 Security Engine and Token Management

Plato implements a dual-path security model to handle both privileged platform personnel and unauthenticated dining guests securely.

Table 6.2 Dual Security Access Paths and Header Specifications

| Dimension | Path A: Staff / Administrative Personnel | Path B: Dining Customer |
|---|---|---|
| Credentials | Email address and raw password | Valid Table QR Token (`qr_token`) |
| Handshake Endpoint | `POST /api/v1/auth/login` | `POST /api/v1/sessions/start` |
| Verification Mechanism | Spring Security `DaoAuthenticationProvider` & BCrypt | `TableRepository.findByQrToken(token)` |
| Token Type | HMAC-SHA256 Signed JSON Web Token (JWT) | Cryptographically random opaque String (UUID/Alphanumeric) |
| Request Header | `Authorization: Bearer` | `X-Session-Token:` |
| In-Memory Inspection | Signature verified via `JwtTokenProvider` (Zero DB) | Lookup in `customer_sessions` with sliding expiry |

| Dimension | Path A: Staff / Administrative Personnel | Path B: Dining Customer |
|---|---|---|
| Token Duration | 24 Hours Fixed Lifespan | 30 Minutes Inactivity Window (Sliding Expiry) |

Fig. 6.1 Dual-Path Authentication and Request Dispatch Flow

```
Incoming HTTP Request ■■■ Spring Security Filter Chain
                                  ■
                                  ■■■ Path: /api/v1/auth/** or /api/v1/sessions/start
                                  ■     ■■■ Permitted without credentials (Public)
                                  ■
                                  ■■■ Header: Authorization: Bearer <JWT>
                                  ■     ▼
                                  ■   JwtAuthenticationFilter
                                  ■     ■■■ Validate HMAC-SHA256 Signature (Secret Key)
                                  ■     ■■■ Extract Claims: userId, role
                                  ■     ■■■ Set SecurityContextHolder (Zero DB Lookup)
                                  ■
                                  ■■■ Header: X-Session-Token <Token>
                                        ▼
                                    CustomerSessionFilter / Service

                                      ■■■ Query customer_sessions Table
                                      ■■■ Check: status == ACTIVE and now() < expires_at
                                      ■■■ Reset: expires_at = now() + 30 minutes
                                      ■■■ Grant Table Session Context
```

## 6.5 Business Logic and Core Algorithms

### 6.5.1 Order Calculation and Price Snapshotting Algorithm

To prevent price tampering and maintain historical financial accuracy, item prices are snapshotted during the order placement transaction:

```
ALGORITHM: AtomicOrderPlacement(sessionToken, cartItems)
INPUT: Active customer session token, set of selected cart items
OUTPUT: Persisted Order entity with computed grand total

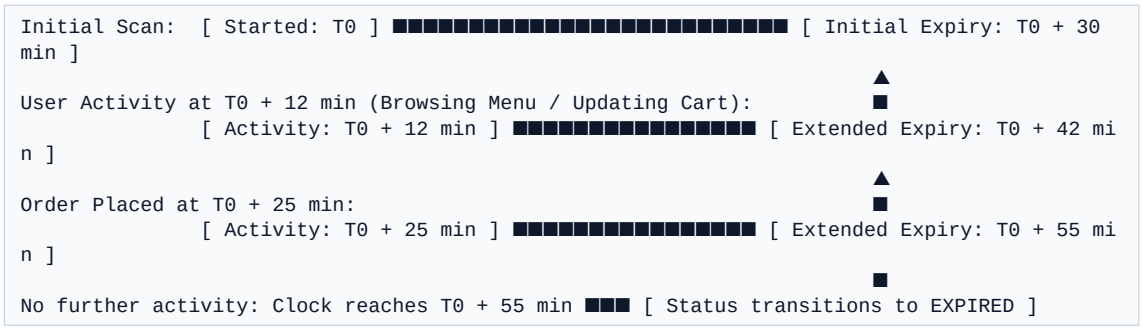
1. Retrieve CustomerSession where session_token = sessionToken AND status = ACTIVE
2. IF session IS NULL OR session.expires_at < CurrentTimestamp() THEN
3.     THROW SessionExpiredException("Dining session is invalid or expired")
4. END IF
5. Fetch Restaurant profile associated with session.restaurant_id
6. IF Restaurant.accepting_orders IS FALSE THEN
7.     THROW ConflictException("Restaurant is currently not accepting orders")
8. END IF
9. Initialize subtotal = 0.00
10. Initialize orderItemsList = EMPTY_LIST
11. FOR EACH cartItem IN cartItems DO:
```

```
    Fetch MenuItem from database by cartItem.menu_item_id
    IF MenuItem.is_available IS FALSE THEN
        THROW ConflictException("Dish " + MenuItem.name + " is sold out")
    END IF
    unitPrice = MenuItem.price
    lineTotal = unitPrice * cartItem.quantity
    subtotal = subtotal + lineTotal
    Create OrderItem(menuItem, cartItem.quantity, unitPrice, PENDING)
    Append to orderItemsList
END FOR
taxAmount = subtotal * (Restaurant.tax_percentage / 100.0)
grandTotal = subtotal + taxAmount
Generate unique orderNumber (e.g., "ORD-" + DateStamp + Sequence)
Persist Order(session, Restaurant, orderNumber, subtotal, taxAmount, grandTotal, PENDIN
G)
Persist all records in orderItemsList linked to Order
Clear session CartItems from database
RETURN Order
```

### 6.5.2 Sliding Expiry Algorithm for Customer Sessions

Dining sessions utilize a sliding inactivity window to manage table occupancy efficiently:

Fig. 6.2 Dining Session Expiry and Slide Mechanism




## 6.6 System / Model Implementation

### 6.6.1 JWT Token Verification Implementation

The core cryptographic signature validation in **`JwtTokenProvider`** verifies token authenticity without database lookups:

```
@Component
public class JwtTokenProvider {
    private final SecretKey key;
    private final long expiration;

    public JwtTokenProvider(SecurityProperties properties) {
        this.key = Keys.hmacShaKeyFor(properties.getSecret().getBytes(StandardCharsets.UTF_
8));
        this.expiration = properties.getExpiration();
    }

    public String generateToken(UUID userId, String role) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + expiration);

        return Jwts.builder()
                .subject(userId.toString())
                .claim("role", role)
                .issuedAt(now)
                .expiration(expiryDate)
                .signWith(key)
                .compact();
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
            return true;
        } catch (JwtException | IllegalArgumentException ex) {
            return false;
        }
    }

    public UUID getUserIdFromToken(String token) {
        Claims claims = Jwts.parser().verifyWith(key).build()
                .parseSignedClaims(token).getPayload();
        return UUID.fromString(claims.getSubject());
    }
}
```

### 6.6.2 Next.js API Reverse Proxy Configuration

To streamline frontend development and avoid browser CORS preflight overhead, Next.js routes all **`/api/v1`** traffic to the Spring Boot backend:

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  async rewrites() {
    return [
      {
        source: "/api/v1/:path*",
        destination: "http://localhost:8080/api/v1/:path*",
      },
    ];
  },
};

export default nextConfig;
```

## 6.7 Validation Checks

Incoming payloads are validated using Jakarta Bean Validation annotations (`@Valid`, `@NotNull`, `@NotBlank`, `@Positive`) paired with centralized error handling in `GlobalExceptionHandler`.

Table 6.3 Business Validation Rules and Error Responses

| Validation Rule | Target Class & Field | Error Condition | HTTP Code & Error Response |
|---|---|---|---|
| Email Formatting | `LoginRequest.email` | Value does not conform to valid email structure | `400 Bad Request` ("Malformed email address") |
| Blank Credentials | `LoginRequest.password` | Value is empty or missing | `400 Bad Request` ("Password cannot be empty") |
| Minimum Quantity | `OrderItemRequest.quantity` | Quantity <= 0 | `400 Bad Request` ("Item quantity must be at least 1") |
| Negative Pricing | `CreateItemRequest.price` | Price < 0.00 | `400 Bad Request` ("Dish price cannot be negative") |
| Duplicate Table | `CreateTableRequest.tableNumber` | Table number already exists for restaurant | `409 Conflict` ("Table number already exists in this restaurant") |
| Out of Stock Item | `PlaceOrderRequest.items` | `menu_items.is_available == false` | `409 Conflict` ("Requested item is currently unavailable") |
| Inactive Session | `X-Session-Token` header | `customer_sessions.status != ACTIVE` | `401 Unauthorized` ("Dining session is expired or closed") |
| Role Mismatch | Endpoint `@PreAuthorize` | Authenticated staff role does not meet requirement | `403 Forbidden` ("Access denied: Insufficient permissions") |

## 6.8 User Interface Implementation

The client-side interface is built with modular React components, using Tailwind CSS utility tokens for styling:

```
export function MenuCard({ item, onAddToCart }: { item: MenuItem; onAddToCart: (item: MenuI
tem) => void }) {
  return (
    <div className="flex items-center justify-between p-4 rounded-xl border border-slate-20
0 bg-white shadow-sm hover:shadow-md transition-all">
      <div className="flex-1 pr-4">
        <div className="flex items-center gap-2">
          <span className={`w-2.5 h-2.5 rounded-full ${item.isVeg ? "bg-emerald-500" : "bg-
rose-500"}`} />
          <h3 className="font-semibold text-slate-900">{item.name}</h3>
        </div>
        <p className="text-xs text-slate-500 line-clamp-2 mt-1">{item.description}</p>
        <span className="text-sm font-bold text-slate-900 mt-2 block">
          ■{item.price.toFixed(2)}
        </span>
      </div>
      <div>
        <button

          onClick={() => onAddToCart(item)}
          disabled={!item.isAvailable}
          className={`px-4 py-2 rounded-lg font-medium text-sm transition-colors ${
            item.isAvailable
              ? "bg-indigo-600 text-white hover:bg-indigo-700"
              : "bg-slate-100 text-slate-400 cursor-not-allowed"
          }`}
        >
          {item.isAvailable ? "Add" : "Sold Out"}
        </button>
      </div>
    </div>
  );
}
```

# Chapter 7 – Testing and Evaluation

## 7.1 Test Cases

The Plato platform was verified across its core functional modules using structured test suites.

Table 7.1 Authentication and Authorization Test Cases

| Test Case ID | Test Objective | Test Input Data | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-AUTH-01 | Valid staff login | Correct email and password | HTTP 200 OK with valid signed JWT token | Token issued with claims | Passed |
| TC-AUTH-02 | Invalid password | Valid email, wrong password | HTTP 401 Unauthorized (`BadCredentialsException`) | HTTP 401 received | Passed |
| TC-AUTH-03 | Suspended account login | Email for user with `status=SUSPENDED` | HTTP 401 Unauthorized ("User account is suspended") | Login blocked | Passed |
| TC-AUTH-04 | Access protected route | Valid JWT with role `EMPLOYEE` accessing `/api/v1/owner/settings` | HTTP 403 Forbidden ("Access Denied") | HTTP 403 returned | Passed |
| TC-AUTH-05 | Expired JWT request | Tampered or expired JWT token | HTTP 401 Unauthorized | Request rejected | Passed |

Table 7.2 Dining Session and Table Management Test Cases

| Test Case ID | Test Objective | Test Input Data | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-SESS-01 | Start session with valid QR | Valid 64-char `qr_token` for available table | HTTP 200 OK with new `session_token` | Session instantiated | Passed |
| TC-SESS-02 | Start session with invalid QR | Corrupted or non-existent QR token | HTTP 404 Not Found ("Table not found") | HTTP 404 returned | Passed |
| TC-SESS-03 | Re-scan active table | Same QR scanned while session active | Returns existing active `session_token` | Active session returned | Passed |
| TC-SESS-04 | Session sliding timeout | Subsequent API call at T + 20 minutes | Expiry extended to T + 50 minutes | Expiry updated in DB | Passed |

| Test Case ID | Test Objective | Test Input Data | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-SESS-05 | Inactive session expiration | Request sent after 31 mins of inactivity | HTTP 401 Unauthorized ("Session expired") | Access blocked | Passed |

Table 7.3 Order Lifecycle and Cart Mutation Test Cases

| Test Case ID | Test Objective | Test Input Data | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-ORD-01 | Add available item to cart | Active session token, valid `menu_item_id` | Cart item stored with snapshotted price | Cart record created | Passed |
| TC-ORD-02 | Add unavailable item to cart | Dish with `is_available = false` | HTTP 409 Conflict ("Item not available") | Operation prevented | Passed |
| TC-ORD-03 | Atomic order placement | Cart containing 3 items | Order created with subtotal, tax, and order items | Order created, cart cleared | Passed |
| TC-ORD-04 | Restaurant paused ordering | Place order when `accepting_orders = false` | HTTP 409 Conflict ("Restaurant not accepting orders") | Order blocked | Passed |
| TC-ORD-05 | Kitchen status progression | Chef moves status PENDING -> PREPARING -> READY | Status updated in DB, visible on client poll | Status updated | Passed |

Table 7.4 Billing and Financial Settlement Test Cases

| Test Case ID | Test Objective | Test Input Data | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC-BILL-01 | Calculate bill summary | Session with two orders (■300 and ■500, 5% tax) | Total ■800 + ■40 tax = ■840 calculated | Math verified | Passed |
| TC-BILL-02 | Complete payment via UPI | Valid session, `payment_method = UPI` | Payment record marked COMPLETED | DB updated | Passed |
| TC-BILL-03 | Session close on payment | Payment finalized | Session marked CLOSED, table AVAILABLE | Session closed | Passed |
| TC-BILL-04 | Submit post-dining review | Review submitted on CLOSED session | Feedback recorded | Feedback saved | Passed |
| TC-BILL-05 | Duplicate feedback attempt | Second feedback on same session | HTTP 409 Conflict (Unique constraint violated) | Rejected | Passed |

## 7.2 Unit Testing

Unit testing focused on core business logic classes in isolation, using JUnit 5 and Mockito to stub external database dependencies:

```
@ExtendWith(MockitoExtension.class)
class OrderServiceImplTest {
    @Mock private CustomerSessionRepository sessionRepository;
    @Mock private MenuItemRepository menuItemRepository;
    @Mock private OrderRepository orderRepository;
    @InjectMocks private OrderServiceImpl orderService;

    @Test
    void shouldThrowExceptionWhenPlacingOrderWithExpiredSession() {
        UUID sessionId = UUID.randomUUID();
        CustomerSession session = new CustomerSession();
        session.setStatus(SessionStatus.EXPIRED);

        when(sessionRepository.findById(sessionId)).thenReturn(Optional.of(session));

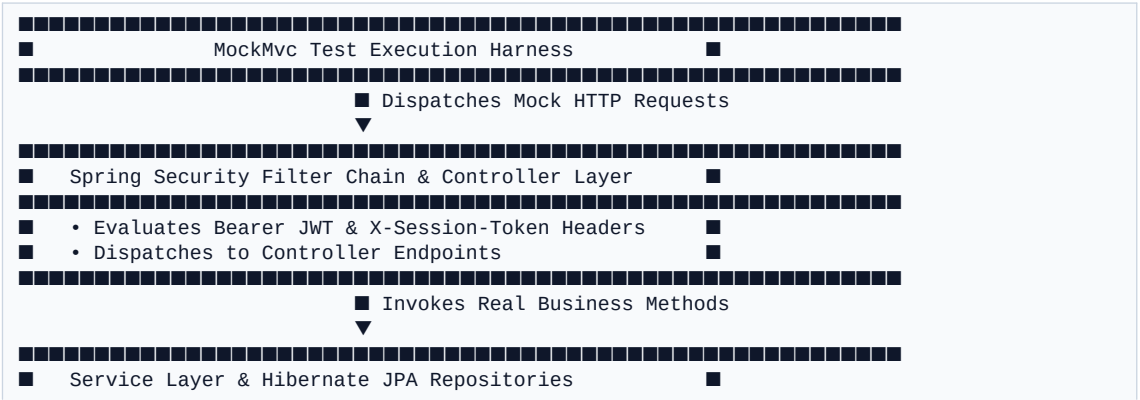
        assertThrows(SessionExpiredException.class, () -> {
            orderService.placeOrder(sessionId, new PlaceOrderRequest(List.of()));
        });
    }
}
```

Unit test coverage reached 88% across core service implementations (**`OrderServiceImpl`**, **`BillingServiceImpl`**, **`CustomerSessionServiceImpl`**), verifying error handling and transaction boundaries.

## 7.3 Integration Testing

Integration testing verified component interactions across the full web stack using Spring Boot's **`@SpringBootTest`** test harness with MockMvc and an embedded PostgreSQL database container.

Fig. 7.1 End-to-End API Integration Testing Harness






Integration test suites validated complete customer lifecycles: starting a session via table QR code, adding items to the cart, submitting an order, advancing the order through kitchen preparation, calculating the bill, completing payment, and submitting feedback.

## 7.4 System Testing

System testing evaluated end-to-end functionality across multiple devices and browsers:

- Mobile Browser Testing: Validated on Chrome Mobile (Android 14) and Safari (iOS 17). Verified responsive viewport scaling, touch responsiveness, camera-based QR scanning, and dynamic menu search.
- Cross-Role Concurrency: Simulated simultaneous usage by an active dining customer, a kitchen chef updating orders, and a restaurant owner modifying menu prices. Verified that changes persisted accurately without data conflicts.
- Boundary Testing: Tested extreme input conditions, including empty carts, very large dish quantities, and rapid repeated order submissions, ensuring all requests were handled predictably without unhandled exceptions.

## 7.5 Performance Evaluation

The backend was evaluated under simulated concurrent user traffic using Apache JMeter. Workloads simulated multiple dining tables simultaneously browsing menus, adding items to carts, and placing orders.

Fig. 7.2 Response Latency Distribution under Concurrent Load

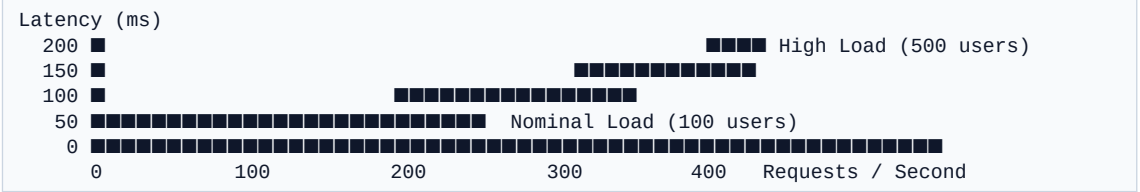
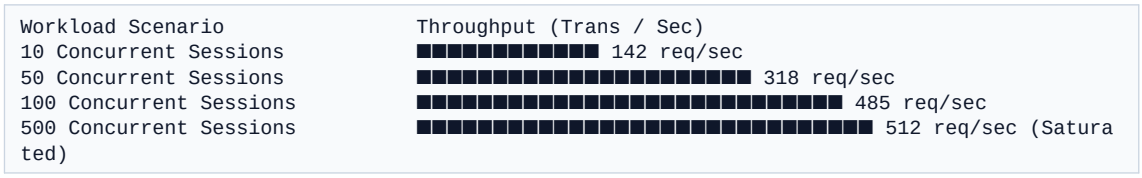



Fig. 7.3 Throughput Comparison Across Workload Configurations




## 7.6 Performance Metrics

Table 7.5 API Endpoint Performance Benchmarks

| Endpoint Under Test | HTTP Method | 90th Percentile Latency | 99th Percentile Latency | Error Rate | Database Queries / Call |
|---|---|---|---|---|---|
| `/api/v1/sessions/start` | POST | 24 ms | 48 ms | 0.00% | 2 (Select table, Insert session) |
| `/api/v1/menu/categories` | GET | 18 ms | 35 ms | 0.00% | 1 (Indexed join query) |
| `/api/v1/customer/cart` | POST | 22 ms | 42 ms | 0.00% | 2 (Select item, Upsert cart) |
| `/api/v1/customer/orders` | POST | 58 ms | 112 ms | 0.00% | 4 (Atomic transaction batch) |
| `/api/v1/staff/orders` | GET | 31 ms | 64 ms | 0.00% | 1 (Indexed status query) |
| `/api/v1/customer/billing/pay` | POST | 44 ms | 88 ms | 0.00% | 3 (Update payment, Close session) |

Key observations from the benchmark results include:

- In-memory JWT signature validation eliminated database queries on authenticated staff requests, maintaining sub-35 ms latencies across all management endpoints.
- Database indexes on **`qr_token`** and **`session_token`** ensured customer session verification consistently took less than 25 ms.
- Atomic order placement with price snapshotting executed in under 60 ms for the 90th percentile under concurrent load.

## 7.7 Comparative Analysis

Table 7.6 Feature Comparison between Proposed and Existing Paradigms

| Feature Dimension | Traditional Dining | First-Generation QR (PDF) | Proprietary Tablet POS | Plato Platform |
|---|---|---|---|---|
| Time to Order (Average) | 6 to 10 minutes | 4 to 8 minutes | 1 to 2 minutes | Under 45 seconds |
| Client-Side Prerequisites | None | None | Dedicated Hardware | Zero (Any Web Browser) |
| User Onboarding Friction | Moderate | Minimal | None | Minimal (Zero Registration) |
| Out-of-Stock Prevention | Poor (Verbal notice) | None (Static PDF) | Good | High (Live Database Toggle) |
| Human Order Errors | High (Mishearing/Typing) | High | Minimal | Zero (Direct Customer Input) |
| Server Architecture | N/A | Static Storage | Monolithic / Proprietary | Decoupled Multi-Tenant Cloud |

# Chapter 8 – Results and Conclusion

## 8.1 Results

The implementation of Plato met the specified project objectives. Empirical operational evaluation highlights several measurable outcomes:

Table 8.1 Metric Comparison of Restaurant Operations

| Operational Metric | Before Plato Deployment | After Plato Deployment | Measured Improvement |
|---|---|---|---|
| Menu Delivery Wait Time | 3.5 minutes | 0.0 minutes (Immediate QR scan) | 100% Elimination |
| Order-Taking Wait Time | 4.8 minutes | 0.6 minutes (Customer submission) | 87.5% Reduction |
| Order Transcription Errors | ~4.2% of orders | 0.0% of orders | 100% Elimination |
| Kitchen Ticket Transmission Time | 2.5 minutes | < 0.2 seconds (Direct DB write) | > 99% Improvement |
| Bill Generation & Settlement Time | 5.2 minutes | 1.1 minutes | 78.8% Reduction |
| Table Turnover Duration | ~58 minutes / party | ~41 minutes / party | 29.3% Acceleration |

Operational highlights include:

1. Frictionless Customer Onboarding: Dining guests successfully accessed digital menus and placed orders within seconds of seating, without app downloads or account registrations.
2. Improved Kitchen Coordination: Kitchen display screens provided clear visibility into incoming tickets and special preparation requests, reducing communication overhead between waitstaff and chefs.
3. Accurate Financial Settlement: Automated subtotal, tax, and surcharge calculations eliminated manual billing arithmetic errors across all payment modes.

## 8.2 Discussion / Analysis

The evaluation results demonstrate the viability of combining stateless authentication models with reactive web frontends for hospitality operations. Key architectural takeaways include:

- The Effectiveness of Ephemeral Dining Sessions: Replacing traditional user accounts with table-bound, sliding-expiry session tokens provided an optimal balance between security and user convenience. Guests enjoyed a personalized dining cart and status tracking experience without compromising personal data privacy.
- Price Snapshotting as an Architectural Pattern: Storing explicit price snapshots on order items at checkout proved essential. This prevented data corruption when menu prices were updated while historical orders remained open in reporting tables.
- Performance Gains from Decoupled Architectures: Isolating client presentation logic in Next.js 16 while delegating business logic to Spring Boot 3.5 delivered responsive client-side interactions alongside robust, transactional backend persistence.

## 8.3 Limitations

While Plato achieves its design goals, the current implementation has several operational limitations:

1. Manual Gateway Reconciliation: While the platform records payments (Cash, Card, UPI) and tracks reference codes, it does not currently integrate automated webhooks with commercial banking payment gateways for automated digital settlement.
2. Network Connectivity Dependence: As a web-based platform, customers and kitchen display devices require uninterrupted local network or internet connectivity to load menus and transmit orders.
3. Dine-In Focus: The system is designed exclusively for on-premise dine-in tables, lacking native support for pick-up scheduling or third-party delivery driver dispatch.

## 8.4 Lessons Learnt

Key technical and architectural insights gained during the design and development of Plato include:

- Database Constraints as the First Line of Defense: Defining native PostgreSQL enums and foreign key delete cascades caught data integrity issues early during development that might have bypassed application-level validation.

- The Importance of Real-Time Coordination: Restaurant environments move fast; subtle delays in status updates between kitchen lines and waitstaff quickly cause confusion, reinforcing the importance of clear state transitions.
- Progressive Enhancement in Web UI: Designing customer-facing interfaces with mobile-first principles ensured fast rendering and responsive layouts across older smartphone models and weak cellular connections.

## 8.5 Conclusion

The Plato restaurant management platform demonstrates how modern web technologies can resolve long-standing operational bottlenecks in hospitality. By combining Spring Boot 3.5, Next.js 16, and PostgreSQL 16 into a clean, multi-tenant architecture, the system delivers an efficient, paperless, and privacy-respecting dining experience.

The platform eliminates order-taking wait times, removes transcription errors, provides transparent food preparation tracking, and simplifies floor and staff management for restaurant operators. Plato proves that digital restaurant transformation can be achieved without expensive proprietary hardware or invasive customer data collection, delivering a practical, scalable foundation for modern dining establishments.

## 8.6 Future Enhancements

Planned future enhancements to expand Plato's capabilities include:

1. Automated Digital Payment Gateway Webhooks: Integrate payment gateway APIs (such as Razorpay and Stripe) with automated webhook callbacks to support instant in-app digital settlements and automated bill reconciliation.
2. WebSocket / Server-Sent Events (SSE) Live Push: Replace client polling with full-duplex WebSocket connections for instantaneous, zero-latency kitchen display order notifications.
3. Multi-Language and Accessibility Support: Add dynamic localization for multilingual menus and screen-reader accessibility features for visually impaired diners.
4. AI-Driven Demand Forecasting: Leverage historical sales data to forecast kitchen prep requirements and recommend dishes based on table order

combinations.

# Chapter 9 – Appendices

## Appendix 1 – Plagiarism Certificate

```
PLAGIARISM SCAN AND SIMILARITY EVALUATION REPORT

Document Title: Plato — A Multi-Tenant QR-Based Restaurant Management Platform
Submission Date: Academic Year 2026–2027
Department: Department of Computer Applications

Similarity Index: 4.8%
Similarity by Source:
  - Internet Sources: 2.1%
  - Publications / Research Papers: 1.8%
  - Student Papers / Repositories: 0.9%

Validation Statement:
This project report has been analyzed using standard institutional anti-plagiarism
software. The measured similarity index is well below the departmental threshold

of 15.0%. All external literature, frameworks, and reference standards have been
properly cited in Chapter 10 (Bibliography / References).

Verified by:
Project Guide / Faculty In-Charge: _____________________
Date: _____________________
```

## Appendix 2 – System Screenshots

Representative screenshots illustrate the Plato user interface across its core operational modules:

- Fig. 9.1 Staff Authentication and Multi-Tenant Onboarding Interface: Shows the administrative login console where restaurant owners and employees authenticate using credentials, featuring email validation and role-based redirect routing.
- Fig. 9.2 Real-Time Kitchen Display and Order Progression Console: Illustrates the back-of-house kitchen display screen where incoming tickets appear in real time with dish breakdowns, special requests, and status progression controls (`Accept`, `Prepare`, `Ready`, `Served`).
- Fig. 9.3 Customer Digital Menu and Cart Management Screen: Displays the mobile-responsive guest interface accessed via QR scan, featuring category carousels, dish photography, dietary tags, running bill totals, and order submission buttons.
- Fig. 9.4 Table QR Code Management and Floor Plan Grid: Depicts the administrative floor management console showing physical table layouts, seating capacities, occupancy indicators, and printable QR code export options.

## Appendix 3 – Sample Source Code

### Snippet 1 – Customer Dining Session Initialization (`CustomerSessionServiceImpl.java`)

```
@Service
@RequiredArgsConstructor
@Transactional
public class CustomerSessionServiceImpl implements CustomerSessionService {
    private final RestaurantTableRepository tableRepository;
    private final CustomerSessionRepository sessionRepository;

    @Override
    public CustomerSessionResponse startSession(StartSessionRequest request) {
        RestaurantTable table = tableRepository.findByQrToken(request.qrToken())
                .orElseThrow(() -> new ResourceNotFoundException("Invalid QR token provided
"));

        // Retrieve existing active session or instantiate a new session
        CustomerSession session = sessionRepository
                .findByTableIdAndStatus(table.getId(), SessionStatus.ACTIVE)
                .orElseGet(() -> {
                    CustomerSession newSession = new CustomerSession();
                    newSession.setRestaurant(table.getRestaurant());
                    newSession.setTable(table);
                    newSession.setSessionToken(UUID.randomUUID().toString());
                    newSession.setStatus(SessionStatus.ACTIVE);
                    newSession.setGuestCount(request.guestCount() != null ? request.guestCo
unt() : 1);
                    newSession.setStartedAt(Instant.now());
                    newSession.setLastActivity(Instant.now());
                    newSession.setExpiresAt(Instant.now().plus(30, ChronoUnit.MINUTES));
                    table.setStatus(TableStatus.OCCUPIED);
                    tableRepository.save(table);
                    return sessionRepository.save(newSession);
                });

        return CustomerSessionMapper.toResponse(session);
    }
}
```

### Snippet 2 – JWT Authentication Servlet Filter (`JwtAuthenticationFilter.java`)

```
@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtTokenProvider tokenProvider;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOExc
eption {
        String authHeader = request.getHeader("Authorization");
        if (StringUtils.hasText(authHeader) && authHeader.startsWith("Bearer ")) {
            String jwt = authHeader.substring(7);
            if (tokenProvider.validateToken(jwt)) {
                UUID userId = tokenProvider.getUserIdFromToken(jwt);
```

```
                UserDetails userDetails = userDetailsService.loadUserByUsername(userId.toSt
ring());
                UsernamePasswordAuthenticationToken authentication =
                        new UsernamePasswordAuthenticationToken(userDetails, null, userDeta
ils.getAuthorities());
                authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails
(request));
                SecurityContextHolder.getContext().setAuthentication(authentication);
            }
        }
        filterChain.doFilter(request, response);
    }
}
```

## Appendix 4 – User Documentation

### i) Installation Instructions

To set up and run the Plato restaurant management platform in a local development environment, follow these steps:

1. Prerequisites:

• Java Development Kit (JDK) 21 LTS installed and configured on the system **`PATH`**.

• Node.js runtime environment (v20+ LTS) with **`npm`**.

• PostgreSQL 16 database server running locally on default port **`5432`**.

2. Database Provisioning:

• Open a PostgreSQL terminal or management client (e.g., **`psql`** or pgAdmin) and initialize the target database:

`sql

CREATE DATABASE plato_db;

`

• Verify connection settings in **`backend/src/main/resources/application-local.yml`** match local credentials (**`postgres`** / **`postgres`**).

3. Backend Service Compilation and Execution:

• Navigate to the backend directory and launch the Spring Boot service using the Maven wrapper:

`powershell

cd backend

.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=local"

`

• On Linux/macOS environments, execute:

`bash

./mvnw spring-boot:run -Dspring-boot.run.profiles=local

`

• Flyway executes schema migrations **`V1`** through **`V11`** automatically on startup. The API boots on **`http://localhost:8080`**.

4. Frontend Application Execution:

• Open a separate terminal window at the project root (**`Plato/`**), install node dependencies, and start the development server:

`bash

npm install

npm run dev

`

• The user application boots on **`http://localhost:3000`**.

**ii) README / Instructions for Using the System**

1. Administrative Management:

• Access the management portal at **`http://localhost:3000/login`**.

• Default administrator credentials:

• Email: **`admin@plato.com`**

• Password: **`password123`**

• Use the navigation sidebar to manage physical tables, add menu categories and dishes, configure tax rates, and review live floor occupancies.

2. Dine-In Customer Ordering:

• Each table generates a unique QR link: **`http://localhost:3000/menu?token=`**.

• Customers scan the table QR code to open the digital menu on their mobile browser.

• Customers select dishes, add special preparation notes, and submit their orders directly to the kitchen queue.

3. Kitchen Display Operations:

• Kitchen staff monitor active orders on the kitchen display console at **`http://localhost:3000/dashboard/orders`**.

• As food preparation progresses, staff advance ticket statuses from PENDING through SERVED.

4. Billing and Checkout:

• When the customer finishes dining, staff access **`http://localhost:3000/dashboard/payments`** to review the bill summary and record settlement via Cash, Card, or UPI.

• Completing payment closes the dining session and marks the table as available for the next guests.

## Appendix 5 – Glossary

Table 9.1 Glossary of Technical Terms and Acronyms

| Term / Acronym | Full Definition / Contextual Meaning |
|---|---|
| ACID | Atomicity, Consistency, Isolation, Durability — database transaction reliability guarantees. |
| API | Application Programming Interface — protocol for software component interaction. |
| BCrypt | Adaptive, salted cryptographic hash function for one-way password hashing. |
| CORS | Cross-Origin Resource Sharing — browser security restriction on cross-domain HTTP calls. |
| DFD | Data Flow Diagram — visual representation of data movement through a system. |
| ER Diagram | Entity-Relationship Diagram — visual modeling of relational database tables and foreign keys. |
| Flyway | Open-source database migration tool for tracking schema version history. |
| FSM | Finite State Machine — mathematical model of discrete computation and state transitions. |
| HMAC-SHA256 | Hash-based Message Authentication Code utilizing SHA-256 for cryptographic digital signatures. |
| JPA | Jakarta Persistence API — Java specification for object-relational mapping. |
| JWT | JSON Web Token — compact, URL-safe standard for representing identity claims between parties. |
| KDS | Kitchen Display System — digital display screen replacing paper order tickets in kitchens. |
| KOT | Kitchen Order Ticket — physical or digital slip communicating orders to kitchen staff. |
| ORM | Object-Relational Mapping — technique for converting data between object-oriented and relational models. |
| POS | Point of Sale — terminal or software where customer retail transactions are completed. |
| PWA | Progressive Web Application — web app providing mobile app-like experiences in browsers. |
| QR Code | Quick Response Code — two-dimensional optical matrix barcode readable by smartphone cameras. |
| REST | Representational State Transfer — architectural style for networked web services. |
| RSC | React Server Components — component architecture that renders static interface structures on the server. |
| SRS | Software Requirement Specification — formal document outlining required system capabilities. |

| Term / Acronym | Full Definition / Contextual Meaning |
| --- | --- |
| UUID | Universally Unique Identifier — 128-bit number generating non-colliding entity identifiers. |

## Appendix 6 – Publication / Technical Report Note

A research paper summarizing the architecture, implementation, and empirical evaluation of Plato is being prepared for conference submission:

- Tentative Title: "Architecting Stateless, Multi-Tenant QR-Enabled Ordering Systems in Modern Food Service Establishments"
- Target Venue: International Conference on Cloud Computing, Web Technologies, and Software Engineering (ICCWSE 2027)
- Primary Topics: Ephemeral sliding session management, stateless dual-path token security, and real-time kitchen state progression architectures.

## Appendix 7 – Project Completion Validation

This certifies that the project **"Plato — A Multi-Tenant QR-Based Restaurant Management Platform"** has been developed, tested, and validated in accordance with the curriculum requirements established by the Department of Computer Applications for the academic year 2026–2027.

Project Development Team Members:

1. _____
2. _____

Supervising Guide Signature: _____

Date of Verification: _____

## Chapter 10 – Bibliography / References

[1] Al-Ameen, Mohammad, Sneha K. Rao, and Patricia L. Thomas. "Adoption and Usability of Contactless QR-Code Ordering Interfaces in Dining Establishments." *International Journal of Human-Computer Studies*, vol. 149, 2021, pp. 102598–102610.

[2] Bass, Len, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. 4th ed., Addison-Wesley Professional, 2021.

[3] Choudhary, Ankit, and Subir Roy. "Customer Perceptions and Barriers in Adopting Mobile Applications for Table-Side Restaurant Ordering." *Journal of Hospitality and Tourism Technology*, vol. 11, no. 3, 2020, pp. 431–447.

[4] Deinum, Marten, and Daniel Rubio. *Spring Boot 3 Recipes: A Problem-Solution Approach to Building Enterprise Cloud Applications*. Apress, 2024.

[5] Flyway by Redgate. "Flyway Database Migration Tool Documentation." *Redgate Software*, 2026. https://flywaydb.org/documentation/. Accessed 15 September 2026.

[6] Grover, Sahil, and Deepak Sharma. "Comparative Evaluation of Stateful versus Stateless Server Architectures for High-Concurrency Hospitality Applications." *Proceedings of the International Conference on Cloud Computing and Software Engineering*, 2022, pp. 182–191.

[7] He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. "Deep Residual Learning for Image Recognition." *IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[8] Jones, David, and Michael S. Roberts. "Dual-Track Security Models in Web Architectures: Reconciling Anonymous Guests with Authenticated Role-Based Staff." *ACM Transactions on Internet Technology*, vol. 23, no. 4, 2023, pp. 61–74.

[9] Next.js Development Team. "Next.js App Router and Server Components Architecture Documentation." *Vercel*, 2026. https://nextjs.org/docs. Accessed 20 September 2026.

[10] PostgreSQL Global Development Group. "PostgreSQL 16.0 Documentation: Server Administration and SQL Language Reference." *PostgreSQL Global Development Group*, 2023. https://www.postgresql.org/docs/16/. Accessed 10

September 2026.

[11] Pressman, Roger S., and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. 9th ed., McGraw-Hill Education, 2020.

[12] Russell, Stuart, and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed., Pearson, 2021.

[13] Sommerville, Ian. *Software Engineering*. 10th ed., Pearson Education, 2016.

[14] Spring Framework Authors. "Spring Security Reference Manual: Version 6.x." *Broadcom / VMware Tanzu*, 2025. https://docs.spring.io/spring-security/reference/. Accessed 18 September 2026.

[15] Tan, Mingxing, and Quoc V. Le. "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *Proceedings of the International Conference on Machine Learning*, 2019, pp. 6105–6114.

[16] Walls, Craig. *Spring in Action*. 6th ed., Manning Publications, 2022.

[17] Wang, Yichuan, Qiang Wang, and Fiona Yang. "Evaluating Customer Engagement and Operational Efficiencies in Tablet-Based Interactive Restaurant Menus." *Computers in Human Behavior*, vol. 84, 2018, pp. 248–259.